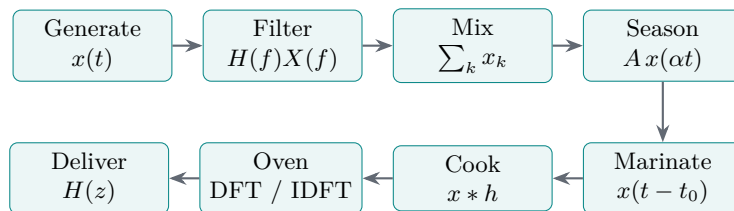

WaveKitchen / WaveBakery

Detailed Gameplay Overview

How the game plays, which files run each part, and how every station maps to Signals & Linear Systems



Contents

1	The game in one page	2
2	How the project is built	2
2.1	Two halves	2
2.2	Most important files	2
2.3	A signal inside the frontend	3
3	The player's journey	4
3.1	Screens around the kitchen	4
3.2	Difficulty and the timer	5
4	Station by station	5
4.1	Pantry and Generate Signal — ingredients are signals	5
4.2	Filtering Lab — washing is filtering	6
4.3	Mixing Lab — the bowl is superposition	7
4.4	Seasoning Lab — amplitude and time scaling	7
4.5	Marinating Lab — time shift	8
4.6	Cooking Lab — convolution with an appliance	8
4.7	Check Dish — hear the defect	9
4.8	Precision Oven — remove the burnt overtone	9
4.9	System Delivery — reject the road vibration	10
4.10	Score, Complete and the leaderboard	10
4.11	Signal Playground (Labs) — the whole game as a sandbox	11
5	The recipes	11
5.1	Chef progression	11
6	Alignment with Signals & Linear Systems	12
7	Parts with little or no impact (flagged)	12
8	Glossary	13

1 The game in one page

WaveKitchen (the frontend calls itself *WaveBakery*) is a cooking game in which every culinary action is a signal-processing operation. Each **ingredient is a signal**, each **kitchen station is a system** that transforms the signal, and the **finished dish is the output** of the whole chain. The player's goal is to make their dish match a hidden *reference dish*: the output of the same chain run with the recipe's ideal parameters on clean ingredients.

Kitchen idea	Signal-processing meaning
Ingredient	A signal $x(t)$ (a periodic waveform, a parametric curve, or a recorded sound)
Dirty ingredient	Signal plus additive noise $x(t) + n(t)$
Washing	Frequency-selective filtering (FFT $\rightarrow$ mask $\rightarrow$ IFFT)
Mixing bowl	Superposition $\sum_k x_k(t)$ (linearity)
Seasoning	Amplitude scaling $Ax(t)$ and time scaling $x(\alpha t)$
Marinating	Time shift (delay) $x(t - t_0)$
Cooking appliance	An LTI system with impulse response $h[n]$; cooking is convolution $y = x * h$
Precision Oven	Find and notch the burnt overtone: sampling theorem, DFT/FFT, IDFT reconstruction
Delivery cart	Reject the road vibration with $H(z)$: poles, zeros, stability, aliased sensing
Tasting table	Similarity metrics between the player's dish and the reference dish

A run is **timed** (5:00 on Easy down to 1:00 on MasterChef), stations are **unlocked in order**, and at the end the dish is **scored**, turned into **stars and points**, and placed on a **leaderboard**.

2 How the project is built

2.1 Two halves

Frontend (what the player sees and hears)

React 19 + TanStack Router/Start, built with Vite, written in TypeScript. It synthesises every ingredient signal mathematically, runs the whole per-station pipeline in the browser, draws every graph, plays every sound through the Web Audio API, and keeps the run state in `localStorage`. Folder: `frontend/src`.

Backend (the authoritative judge)

FastAPI + SQLAlchemy + SQLite, with NumPy/SciPy for DSP. It stores chefs, sessions, attempts and the leaderboard. On submission it *re-builds the dish from scratch* from the recorded parameters (it never trusts audio sent by the client) and scores it against the reference dish. Folder: `backend/app`.

Runner

`run.py` starts both servers with one command (backend on port 8000, Vite on port 5173, which proxies `/api` to the backend).

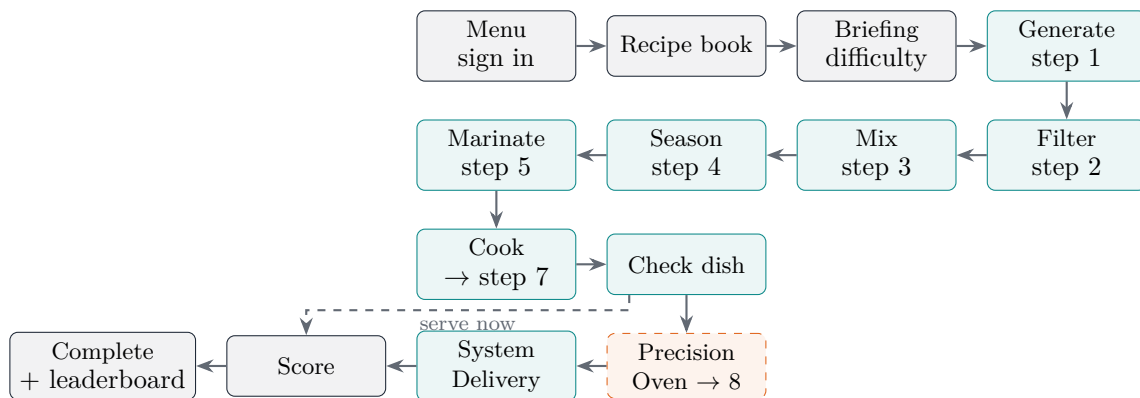
2.2 Most important files

File	Role
<code>frontend/src/lib/signals.ts</code>	Every ingredient's mathematical signal (equations, parametric curves, the recorded chicken), and the mixing-bowl superposition curve.
<code>frontend/src/lib/pipeline.ts</code>	The per-station DSP pipeline: mixing, seasoning, marinating, convolution, the reference dish, curve carrying, stage storage and invalidation.
<code>frontend/src/lib/dsp.ts</code>	FFT, low-pass filtering, convolution, and the similarity metric used for the final score.
<code>frontend/src/lib/precision-oven-dsp.ts</code>	Sampling/aliasing analysis, radix-2 FFT/IFFT, oven equaliser, reconstruction and oven metrics.
<code>frontend/src/lib/z-system.ts</code>	Delivery-cart systems: poles, zeros, gain, difference equation, stability score.
<code>frontend/src/lib/cooking-audio.ts</code>	The four cooking impulse responses $h[n]$ (grill, fry, bake, boil).
<code>frontend/src/lib/audio.ts</code>	Turns a 401-sample signal into audible sound.
<code>frontend/src/lib/recipes.ts</code>	Recipe book, difficulty/timer, run session, progression, backend sync and submission.
<code>frontend/src/lib/leaderboard.ts</code>	The on-device leaderboard.
<code>frontend/src/lib/api.ts</code>	HTTP client for the backend.
<code>frontend/src/routes/*.tsx</code>	One file per screen/station (Section 3).
<code>backend/app/dsp/core.py</code>	Backend DSP primitives: FFT, masks, filters, convolution, time operators, modulation, decimation.
<code>backend/app/dsp/instruments.py</code>	Backend ingredient synthesis (instrument-like voices).
<code>backend/app/dsp/contamination.py</code>	Seeded noise that makes ingredients “dirty”.
<code>backend/app/dsp/systems.py</code>	Appliance impulse responses (comb, decay, resonator, moving average, differencer).
<code>backend/app/dsp/pipeline.py</code>	The canonical backend pipeline <code>run_pipeline</code> .
<code>backend/app/dsp/metrics.py</code>	Prep (cleaning) score and dish score.
<code>backend/app/gameplay.py</code>	Judging, points, ranks, tier unlocks, System Delivery score.
<code>backend/app/routers/*.py</code>	HTTP endpoints: players, sessions, leaderboard, catalogue, direct DSP.
<code>backend/app/seed.py</code>	Ingredient, appliance and recipe catalogue (with targets).

2.3 A signal inside the frontend

Every stage passes around a `PipelineSignal` (defined in `frontend/src/lib/pipeline.ts`): **401 samples** spanning a *normalised one-second window*. Frequencies inside the game are therefore “game hertz”: cycles per window. A 4 Hz ingredient shows 4 cycles on screen. When a signal is played, `frontend/src/lib/audio.ts` repeats the window fast enough that its fundamental becomes an audible pitch between 110 and 880 Hz, so what you hear has the same shape as what you see.

3 The player's journey



The dashed orange box is optional (a dish can be served straight from Check Dish). Numbers are the progression levels stored in `localStorage` (`wavebakery_progress_<recipe>`): a station stays locked (“Station Locked” screen) until the previous level is reached.

3.1 Screens around the kitchen

Screen	File	What happens
Landing / Menu	<code>routes/index.tsx</code> , <code>routes/menu.tsx</code>	Entry point; opens the chef sign-in (<code>components/game/ChefAuthModal.tsx</code>), leaderboard, settings, how-to-play and the Signal Playground.
Chef sign-in	<code>components/game/ChefAuthModal.tsx</code>	Register or sign in with name + password; the backend returns a token that identifies the chef on every later request.
Recipe book	<code>routes/recipe-book.tsx</code>	Choose one of 5 recipes; shows tier, difficulty and best score.
Briefing	<code>routes/briefing.tsx</code>	Shows the recipe's ingredients, and lets the player choose difficulty (timer). Starting calls <code>startRecipeRun</code> , which resets progress, creates the run session and (if signed in) a backend game session.
Kitchen map / hub	<code>routes/kitchen.tsx</code> , <code>routes/kitchen-hub.tsx</code>	Station map and navigation hub.
Pantry	<code>routes/pantry.tsx</code>	Browse every ingredient and inspect its equation, waveform and spectrum (<code>components/game/IngredientSignalModal.tsx</code>).
Settings / How to play	<code>routes/settings.tsx</code> , <code>routes/how-to-play.tsx</code>	Theme, chef name, logout; the tutorial.
Leaderboard	<code>routes/leaderboard.tsx</code>	Server rankings per recipe/difficulty, global chef ranking, kitchen analytics, plus the on-device list when offline.

3.2 Difficulty and the timer

Difficulty	Time	Score multiplier	Source
Easy	5:00	$\times 0.8$	DIFFICULTY_CONFIGS in <code>lib/recipes.ts</code> ,
Medium	3:30	$\times 1.0$	multipliers in <code>routes/score.tsx</code>
Hard	2:00	$\times 1.25$	
MasterChef	1:00	$\times 1.5$	

The timer (`useRecipeTimer`) counts down from the run's start. When it hits zero an expiry dialog (`components/game/RecipeTimer.tsx`) blocks the run and offers a retry; no completion score is awarded for an expired run. Time left at the end becomes a bonus (Section 4.10).

4 Station by station

Each station below lists what the player does, the files that implement it, the Signals & Systems concept it teaches, and why it exists and what it changes in the final result.

4.1 Pantry and Generate Signal — ingredients are signals

What the player does. Picks ingredients for the recipe, sees each ingredient's equation, waveform and spectrum, and listens to it. Finishing unlocks Filtering (or Mixing directly if nothing needs washing).

Files involved

`frontend/src/routes/generate.tsx` (station), `frontend/src/routes/pantry.tsx`, `frontend/src/lib/signals.ts` (all ingredient models), `getRecipeIngredientSamples` in `frontend/src/lib/pipeline.ts`, `frontend/src/lib/chicken-audio.ts` and `frontend/src/lib/chicken-samples.ts` (recorded chicken), `frontend/src/lib/audio.ts`.

The ingredients form a small catalogue of textbook signal families:

Ingredient	Family	Model (from <code>lib/signals.ts</code>)
Cheese	triangle	$x(t) = A \frac{2}{\pi} \arcsin(\sin(2\pi ft + \phi))$ — odd harmonics, $1/n^2$ roll-off
Sugar	square	$x(t) = A \operatorname{sgn}(\sin(2\pi ft + \phi))$ — odd harmonics, $1/n$ roll-off
Salt	square	same as sugar with small amplitude $A_s = 0.4$
Bread	superposition	$y(t) = 0.8 \sin(6\pi t) + 0.6 \sin(18\pi t)$ (3 Hz + 9 Hz)
Milk	sum of sines	$y = 1.4 \sin(0.7x) + 0.6 \sin(1.3x) + 0.3 \sin(2.1x + 0.8)$ (non-harmonic)
Flour	“flame”	$\operatorname{sq}(t) - \operatorname{sq}(2t) \cos(2\pi t)$ (product of signals)
Bun	dome	$5(0.5 + 0.5 \cos 0.3x)^{0.25} + 0.03 \cos 9x$
Butter	trapezoid	$\max(0, 1.2 - 1.2 \max(\operatorname{mod}(x + 9, 14) - 9 - 4, 0))$
Carrot	taper	$5 - (1 + \cos 0.5x)^4$
Cucumber	saturated	$6 \tanh(4 \cos x)$ — a soft-clipped cosine
Beef patty	parametric	$x = 2.5 \cos t + 5 \lfloor t/2\pi \rfloor, y = 0.7 \sin t$
Lettuce	parametric	$x = t, y = 2.2 \cos t + 0.45 \cos 7.5t$

Ingredient	Family	Model (from <code>lib/signals.ts</code>)
Noodles	parametric	$x = t + 3 \sin t$, $y = 5 \cos t$ (trochoid loops)
Tomato	parametric	$x = (2 + 0.2 \sin 3t) \cos t$, $y = -(1.5 + 0.2 \sin t) \sin t$
Onion	parametric	$x = (0.1 + 0.08t) \cos 3t$, $y = (0.1 + 0.08t) \sin 3t$ (spiral)
Sauce	parametric	$x = (3 + 2 \sin 5t) \cos t$, $y = (2 + \sin 5t) \sin t$
Egg	parametric	$x = (1.5 + 0.5 \sin t) \cos t$, $y = -(2 + 0.2 \sin t) \sin t$
Chicken	recorded	$s(t) = \text{PCM samples of public/sounds/chicken.wav (44.1 kHz)}$

A *parametric* ingredient is a 2-D curve $(x(t), y(t))$. For sound and for the 1-D pipeline the game uses its vertical component $y(t)$ as the time signal; for the mixing-bowl picture it keeps the full curve.

Signals & Systems concept

Continuous-time signal models and their classification: periodic vs aperiodic, sinusoids, square/triangle waves and their Fourier-series harmonic content, superposition of sinusoids, nonlinear shaping (saturation, clipping), parametric curves, and a real sampled recording (discrete-time signal at 44.1 kHz).

Why it is here & its impact

Every later station operates on these signals, so this is the input to the whole system. Choosing ingredients decides what goes into the bowl; a wrong or missing ingredient changes the mix and lowers the Mixing accuracy and the final similarity. Different families were chosen so the player can *see* why a square wave has a richer spectrum than a sine and why filtering affects them differently.

4.2 Filtering Lab — washing is filtering

What the player does. Each *washable* ingredient arrives with noise. The player drags a **cutoff frequency** slider over the spectrum, applies the filter and must reach a cleanliness of at least 85 % before moving on.

Files involved

```
frontend/src/routes/filtering.tsx; applyLowPassFilter, fft in
frontend/src/lib/dsp.ts; backend apply_filters / accept_ingredient in
backend/app/routers/sessions.py; dirty_signal, filtered_signal in
backend/app/gameplay.py; backend/app/dsp/contamination.py; apply_filter_chain in
backend/app/dsp/pipeline.py; prep_quality in backend/app/dsp/metrics.py.
```

When a backend session exists, the Filtering Lab shows the *server's* real dirty and clean signal for that ingredient, sends the chosen cutoff to the server (`filterIngredient`), and displays the server's filtered result and cleaning score. Offline, a local approximation is used. The filtered samples are saved and become the ingredient's input to Mixing.

Signals & Systems concept

Additive noise, the Fourier transform (FFT), the frequency domain, ideal vs practical low-pass filters (a raised-cosine transition band), filtering as multiplication in frequency $Y(f) = H(f)X(f)$, and the trade-off between removing noise and keeping the signal's own frequencies (cutting too low distorts the ingredient).

Why it is here & its impact

It is the first place the player must reason in the frequency domain. The server's cleaning ("prep") score scales the authoritative dish score (it is worth up to 28 %) and is reported as the Filtering sub-score; the local filtering accuracy is one of the stage averages in the offline score. Over-filtering is detected and named in the feedback ("you cut real ingredient frequencies").

4.3 Mixing Lab — the bowl is superposition

What the player does. Drags ingredients from the tray into the bowl and presses *Mix*. The oscilloscope shows each ingredient and the combined signal.

Files involved

```
frontend/src/routes/mixing.tsx; computeMixedSignal, withMixCurve,
getMixingIngredientSamples in frontend/src/lib/pipeline.ts;
computeSuperpositionCurve, computeSuperpositionPath in
frontend/src/lib/signals.ts; backend bowl_signals, compute_stages in
backend/app/gameplay.py and stage_mix in backend/app/dsp/pipeline.py.
```

For ordinary ingredients the mix is the sample-by-sample sum. When a parametric ingredient is present the bowl draws the *2-D geometric* superposition $(\sum_k x_k(t), \sum_k y_k(t))$, which is why curves such as noodles make loops. That curve is carried as a visual track through every later station. The bowl contents are sent to the server, which mixes exactly those ingredients.

Signals & Systems concept

Linearity and the superposition principle; normalisation of a sum $(1/\sqrt{K})$ to keep amplitude comparable; the difference between adding signals in time and adding curves in the plane.

Why it is here & its impact

Mixing accuracy is the fraction of recipe ingredients correctly placed. On the server, missing or extra ingredients change the mixed signal and therefore the Mixing sub-score and every stage after it.

4.4 Seasoning Lab — amplitude and time scaling

What the player does. Sets *Signal strength* (amplitude A) and *Frequency character* (α) and watches the player curve approach the dashed target curve. Accuracy $\geq 85\%$ completes the station.

Files involved

```
frontend/src/routes/transform.tsx; computeSeasonedSignal, timeScaleSamples,
transformCurve in frontend/src/lib/pipeline.ts; backend stage_season and
stage_blend (time_scale) in backend/app/dsp/pipeline.py and
backend/app/dsp/core.py.
```

Signals & Systems concept

Amplitude scaling $y(t) = Ax(t)$ and time scaling $y(t) = x(\alpha t)$: $\alpha > 1$ compresses the signal (higher frequencies, more cycles), $\alpha < 1$ expands it. Time scaling scales the spectrum: $x(\alpha t) \leftrightarrow \frac{1}{|\alpha|}X(f/\alpha)$.

Why it is here & its impact

The dials are synced to the server (as `seasoning` and `blend`); a wrong gain or rate changes the reference comparison and triggers named feedback (“over-seasoned”, “over-blended”). The seasoning accuracy is a stage average and half of the “Transformation” score.

4.5 Marinating Lab — time shift

What the player does. Chooses a marinating time; the waveform is delayed (a flat lead-in, then the signal) and must match the target timing.

Files involved

`frontend/src/routes/marinate.tsx`; `computeMarinatedSignal`, `delayCurve` in `frontend/src/lib/pipeline.ts`; backend `stage_marinate` and `time_shift` (fractional delays via a linear phase ramp) in `backend/app/dsp/pipeline.py` and `backend/app/dsp/core.py`.

Signals & Systems concept

Time shifting $y(t) = x(t - t_0)$; causality and zero padding; the shift theorem $x(t - t_0) \leftrightarrow X(f)e^{-j2\pi f t_0}$, which the backend uses to delay by non-integer numbers of samples.

Why it is here & its impact

The time is synced to the server as `marinate` (seconds). It completes the Transformation score together with seasoning, and the server names mistakes (“marinated too long — excess time shift”).

4.6 Cooking Lab — convolution with an appliance

What the player does. Picks a cooking method (grill, fry, bake, boil — each is an impulse response) and slides the impulse across the signal to set the convolution depth. The panels show input x , impulse response h , output $x * h$ and the target.

Files involved

`frontend/src/routes/cooking.tsx`; `computeConvolvedSignal` in `frontend/src/lib/pipeline.ts`; kernels in `frontend/src/lib/cooking-audio.ts`; backend `stage_cook`, `apply_system`, `cascade` and the appliance impulse responses in `backend/app/dsp/systems.py`.

Signals & Systems concept

Linear time-invariant (LTI) systems are fully described by the impulse response; the output is the convolution $y[n] = \sum_k x[k] h[n - k]$. Different h give different characters: a comb of echoes (grill), a bright decaying burst (fry), a smooth long tail / low-pass (bake), a damped resonator (boil). A cascade of LTI systems is one LTI system with $h = h_1 * h_2 * \dots$.

Why it is here & its impact

The chosen appliance is the heart of the dish. The server’s Cooking sub-score now isolates this choice: it compares the player’s pre-cooking signal through the recipe’s appliances against the same signal through the player’s appliances, so the right appliance scores 100 even if earlier

stages were imperfect.

4.7 Check Dish — hear the defect

What the player does. Inspects and listens to the dish *as it leaves the kitchen*: the cooked dish plus the **burnt overtone** cooking left in it (a recipe-specific tone between two harmonics of the dish). Then either serves it now — the overtone stays in the dish — or takes it to the Precision Oven to remove it (`components/game/ServeChoiceModal.tsx`).

Files involved

`frontend/src/routes/check-dish.tsx`, `servedDish` in `frontend/src/lib/delivery.ts`.

Why it is here & its impact

The decision point for the finishing stations: serving now keeps the defect, which lowers the similarity to the target on the score screen and on the server.

4.8 Precision Oven — remove the burnt overtone

What the player does. Five steps:

1. **Sampling**: pick a sampling rate f_s and watch the reconstruction; below $2f_{\max}$ the dish aliases. The overtone is part of the dish, so it raises $f_{\max}$.
2. **Unlock**: type the minimum safe rate; it must satisfy $f_s \geq 2f_{\max}$ and be at most 64 Hz.
3. **FFT lab**: compare the dish's spectrum with the target's. The target has no overtone, so a spike stands out; remove it with the charred-notch filter (or the browning cutoff, which also removes real harmonics) and balance the three bands.
4. **IFFT**: rebuild the dish from the edited spectrum and listen.
5. **Plate**: the rebuilt dish becomes the delivered dish.

Files involved

`frontend/src/routes/beam-delivery.tsx` (the Precision Oven screen; the file name is historical), `frontend/src/lib/precision-oven-dsp.ts`, `frontend/src/lib/delivery.ts`; backend `apply_oven` in `backend/app/delivery.py`.

Signals & Systems concept

Spectral diagnosis (finding an unwanted component in the DFT), notch vs low-pass filtering and their side effects, the Nyquist–Shannon sampling theorem and aliasing, the DFT axis $f_k = kf_s/N$, zero padding, the radix-2 FFT, Hermitian symmetry, the inverse DFT and band-limited (Dirichlet) reconstruction.

Why it is here & its impact

The rebuilt dish *is* what gets served, so the oven changes the final similarity directly. Leaving the overtone scores about 50–70; notching it scores about 90–98. The server receives the oven *settings* and applies the same filter to its own dish.

4.9 System Delivery — reject the road vibration

What the player does. The road adds a **vibration tone** (recipe-specific frequency) to the dish on the cart. The player chooses the cart’s suspension (1st-order low-pass, 2nd-order resonator, moving average, notch), places the pole or zero on the z-plane (radius r , angle ω_0) and picks the vibration sensor’s sampling rate. An orange marker shows where the sensor reports the vibration on $|H(e^{j\omega})|$; the cart’s filtered output is what gets served (“Listen equalized dish”).

Files involved

```
frontend/src/routes/system-delivery.tsx,
frontend/src/components/system/SystemResponsePlotter.tsx,
frontend/src/lib/z-system.ts, deliverOnCart in frontend/src/lib/delivery.ts;
backend deliver_on_cart in backend/app/delivery.py.
```

Signals & Systems concept

Filter design with poles and zeros: a zero on the unit circle at the disturbance frequency (notch) removes it; a low-pass pole trades vibration removal against dulling the dish; poles outside $|z| = 1$ make the system unstable (BIBO). Aliasing: a sensor sampling below twice the vibration frequency reports a folded frequency, so a notch aimed at that reading misses the real tone.

Why it is here & its impact

The cart’s output is the final dish. A notch on the real vibration scores about 92–98, a tuned low-pass about 85–93, no filtering or the default about 50–80, a notch aimed at the aliased reading about 65, and an unstable cart below 50. The server rebuilds the same vibration and filter from the settings.

4.10 Score, Complete and the leaderboard

What the player sees. The score screen compares the player’s dish with the target (waveforms and a similarity percentage), shows the stage breakdown, stars, Chef Fourier’s feedback and a customer critique, and the total points. The Complete page repeats the score and stars; the leaderboard ranks the run.

Files involved

```
frontend/src/routes/score.tsx, frontend/src/routes/complete.tsx,
frontend/src/lib/critiques.ts, frontend/src/lib/leaderboard.ts,
frontend/src/routes/leaderboard.tsx; backend submit in
backend/app/routers/sessions.py, judge, award, maybe_unlock in
backend/app/gameplay.py, backend/app/dsp/metrics.py,
backend/app/routers/leaderboard.py.
```

How it works in short:

1. The latest parameters are synced, then the run is submitted once.
2. The server rebuilds the player’s dish and the reference dish, computes a 0–100 score and stars (92/80/66/50/32 for 5–1 stars), points and possible tier unlocks. Offline, the browser computes a 0–100 score with the same weighting.
3. Total points on screen = score $\times 10 \times$ difficulty multiplier + a time bonus (2 per second left, at most 300, also $\times$ difficulty).

- The run is saved to the on-device leaderboard; the server leaderboard keeps each chef's best dish per recipe (filterable by difficulty).

Signals & Systems concept

Signal similarity measures: normalised cross-correlation, (normalised) mean-square error and SNR in dB, and spectral similarity (cosine similarity of dB magnitude spectra).

Why it is here & its impact

This is where every earlier decision becomes a number; the full formulas are in the *Detailed Architecture* document.

4.11 Signal Playground (Labs) — the whole game as a sandbox

`frontend/src/routes/labs.tsx` runs every station of the game on any ingredient or oscillator, with no timer and no locked stages, using the recipe stations' own functions (`frontend/src/lib/playground.ts`): generator, washing (low-pass), mixing (superposition), seasoning ($A \cdot x(\alpha t)$), marinating (time shift $x(t - t_0)$), cooking (convolution, with the cooking sound while the depth slider is held), the **Precision Oven** (burnt overtone, sampling at f_s with Nyquist and aliasing, FFT, band gains, cutoff and notch, IFFT), **System Delivery** (road vibration, the station's z-plane plotter, $|H(e^{j\omega})|$, stability and the aliasing sensor), **Tasting** (the live match and the server's dish score, part by part) and **Full Chain** (a whole recipe station by station; at the recipe's own dials it reproduces the reference dish). The labs live in `frontend/src/components/playground/*`. Nothing here affects a recipe run or the score; it exists for exploration and teaching.

5 The recipes

Recipe	Tier	Ingredients	Season A, α	Marin.	Cook
Burger	1	Bun, Beef Patty, Cheese, Lettuce, Tomato, Salt	1.5, 0.8	1.25	grill
Sandwich	1	Bread, Chicken, Cheese, Lettuce, Tomato, Salt, Sauce	1.2, 1.1	0.85	grill
Cake	2	Flour, Egg, Butter, Sugar, Milk	1.8, 0.6	1.5	bake
Noodles	2	Noodles, Egg, Chicken, Onion, Salt	1.3, 1.2	1.75	boil
Chicken Fry	3	Chicken, Flour, Egg, Salt, Butter	2.0, 1.4	1.4	fry

Targets live in `frontend/src/lib/recipes.ts` and, for the server, `backend/app/seed.py`; the two are kept identical.

5.1 Chef progression

- Points:** a served dish earns $\lfloor \text{score} \times \text{tier} \times 1.5 \rfloor$ points, but only if it beats the chef's previous best on that recipe, and only on an unlocked tier (locked-tier runs are ranked but earn nothing).
- Ranks** (by total points): Dishwasher 0, Prep Cook 250, Line Cook 600, Sous Chef 1100, Head Chef 1800, Signal Master 2600.
- Tiers:** clearing 2 recipes of the current tier with a score ≥ 60 unlocks the next tier.

6 Alignment with Signals & Linear Systems

Concept	Where	Why it was added	Impact on the game
Signal classification, periodic signals, harmonics	Pantry, Generate	Ingredients must look and sound different for reasons the player can explain	Defines every input; wrong choice lowers Mixing
Fourier transform / spectrum	Filtering, Oven, Delivery	Noise is easier to see and remove in frequency	Drives the cleaning score (scales the server score, up to 28 %)
Filtering ($H(f)X(f)$)	Filtering	Washing = removing the noise band without cutting the ingredient	Prep score, over/under-filter feedback
Linearity / superposition	Mixing	A bowl is literally a sum	Mixing accuracy and server mix
Amplitude and time scaling	Seasoning	Two dials that change height and rate	Transformation score, server params
Time shift	Marinating	Delay is the simplest system	Transformation score, server params
Sampling and aliasing	Oven, Delivery	Nyquist: sample fast enough, or the dish (and the vibration sensor) alias	Oven score; cart aimed at the aliased reading misses
LTI systems, impulse response, convolution	Cooking	Each appliance is a system h	Central to the dish; Cooking score
DFT/FFT, notch filtering, IDFT, reconstruction	Precision Oven	Cooking leaves a burnt overtone to find and remove	The oven's output is the served dish (about +30 similarity)
z-transform, poles/zeros, stability, aliasing	System Delivery	The road shakes the dish; the cart must reject it	The cart's output is the served dish; unstable ruins it
Correlation, MSE/SNR, spectral similarity	Score	Objective comparison with the target	The final score, stars, leaderboard

7 Parts with little or no impact (flagged)

While mapping the project, the following parts were found to be unrelated to the Signals & Systems content or to have no effect on the game's result.

▲ Flag: low or no impact

1. **Backend “direct DSP” endpoints** (`/api/dsp/generate`, `filter`, `mix`, `transform`, `convolve`, `critique`) and their client methods in `lib/api.ts`: never called by the game.
2. **Unused client methods** in `lib/api.ts`: `getConvolution`, `getHealth`, `getPlayerHistory`, `getRecipes`, `getRecipe`, `getIngredients`, `getAppliances`,

`loginPlayer`, `registerPlayer`.

3. **Unreachable noise types:** `corrupt` in `backend/app/dsp/contamination.py` only ever picks white, hiss or burst noise; its pink, hum and tone branches never run.
4. **WAV impulse-response loader** `loadCookingImpulseAudio` in `frontend/src/lib/cooking-audio.ts`: never called; cooking always uses the static kernels.
5. **Two different ingredient models:** the server synthesises ingredients as instrument-like audio (`backend/app/dsp/instruments.py`) at 22 050 Hz, while the browser uses the mathematical models above. The server score is therefore computed on a different (but parameter-equivalent) dish than the one drawn on screen; only the Filtering Lab shows the server's own signals.

8 Glossary

Game hertz	Cycles per normalised one-second window of 401 samples.
Reference dish	The pipeline output for clean ingredients with the recipe's exact parameters.
Stage accuracy	A 0–100 value each station records on the run session.
Prep score	The server's cleaning score for a filtered ingredient.
Session	The server's record of one run (seed, parameters, status).
Attempt	A served dish on the server; the leaderboard's source.